

Maqsat & Family — Архитектура

Микросервисный бэкэнд для Halyk SuperApp: AI-бюджетирование · цели/накопления · семейный контур с детскими лимитами и SOS-одобрением. • 2026

1. Что это

- **AI-бюджетирование** — доход разбивается на категории; траты авто-категоризируются по MCC и трекаются против бюджета.
- **Цели / Maqsat** — накопления на виртуальных счетах с прогрессом.
- **Family** — общие группы, роли adult/child, дневные лимиты. Ключевой сценарий: ребёнок унёсся в лимит на кассе → SOS-push родителю → одобрение в один тап.

Стек: Java 21 · Gradle (Kotlin DSL) monorepo · Spring Boot 3.4.1 · Spring Cloud 2024.0.0 · Gateway (reactive) · Eureka · PostgreSQL 16 (db-per-service, Flyway) · Kafka 3.9 (KRaft) · Keycloak 26 (OIDC) · Micrometer/OTel → Prometheus/Tempo/Loki/Grafana.

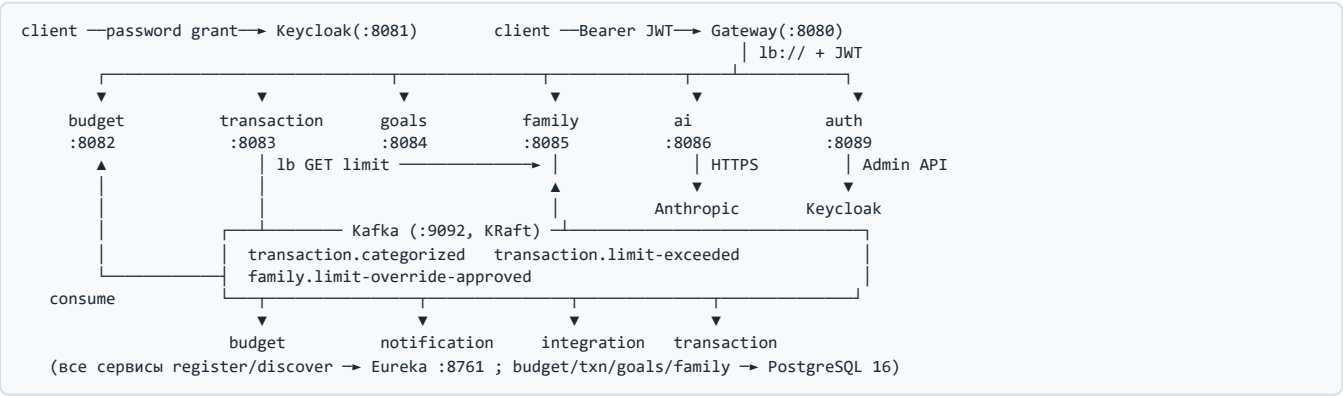
2. Принципы

- **Виртуальный слой** — деньги на основном счёте; лимиты накладываются маской через API, процессинг банка не трогаем.
- **Инвайты без передачи пароля** — приглашённому создаётся временный пароль с обязательной сменой (UPDATE_PASSWORD).
- **Роли в домене** — adult/child в family-service (привязаны к membership), не в Keycloak. Keycloak отвечает только «кто ты» (sub).
- **Тонкий common** — только контракты событий, ApiError, CurrentUser.
- **Состояние через Kafka**; синхронные чтения — REST/WebClient через Eureka (lb://).

3. Сервисы

Сервис	Порт	БД	Назначение	Kafka
gateway	8080	—	Маршрутизация /api/<svc>/** + проверка JWT	—
keycloak	8081	keycloak_db	IdP (OIDC)	—
budget	8082	budget_db	Планы, категории, лимиты, трекинг	consumer
transaction	8083	transaction_db	Транзакции + categorization engine	prod+cons
goals	8084	goals_db	Цели, виртуальные счета	—
family	8085	family_db	Группы, роли, лимиты, SOS-одобрение	producer
ai-assistant	8086	—	LLM-оркестратор (Anthropic)	—
notification	8087	—	Push-имитация / Smart-Push	consumer
integration	8088	—	Бонусы/офферы (stub)	consumer
auth	8089	—	Онбординг/инвайты (Keycloak Admin API)	—
eureka	8761	—	Service discovery	—

4. Схема взаимодействия



5. Контракты событий (JSON, ключ = userId, records в common)

Событие	Топик	Поля
TransactionCategorized	transaction.categorized	transactionId, userId, accountId, amount, mcc, categoryName, occurredAt
LimitExceeded	transaction.limit-exceeded	transactionId, childUserId, accountId, attemptedAmount, dailyLimit, shortfall, mcc, merchant
LimitOverrideApproved	family.limit-override-approved	transactionId, childUserId, approvedAmount, approvedByUserId

6. Конфликтный сценарий (ядро продукта)

1. Ребёнок: POST /api/transactions на 5000 при лимите 3000.
2. transaction читает лимит из **family** (lb WebClient, проброс bearer ребёнка).
3. Превышение → транзакция в статус **PENDING_APPROVAL** (не отклоняется) + publish **LimitExceeded**.
4. **notification** консьюмит → **SOS Smart-Push** родителю «одобрить транзакцию?».
5. Родитель: POST /api/family/approvals/{txnId} → family пишет override в child_limit + publish **LimitOverrideApproved**.
6. transaction консьюмит → проводит платёж (**POSTED**) + publish **TransactionCategorized** (учёт в бюджете).

7. Модель данных (database-per-service, Flyway, id uuid pk)

budget_db

- budget_plan (owner_id, owner_type[USER|GROUP], period_start, period_end, created_at)
- budget_category (plan_id→plan, name, type[MANDATORY|DISCRETIONARY], limit_amount, spent_amount)

transaction_db

- txn (account_id, user_id, amount, merchant, mcc, category_name, occurred_at, status[POSTED|DECLINED|PENDING_APPROVAL])

goals_db

- virtual_account (owner_id, owner_type, balance)
- goal (name, target_amount, allocated_amount, monthly_contribution, deadline, virtual_account_id→va, bonus_program_ref)

family_db

- family_group (name, type='FAMILY', created_by, created_at)
- membership (group_id→group, user_id, role[ADULT|CHILD], unique(group_id,user_id))
- child_limit (membership_id→membership, daily_limit, override_until, override_amount)

8. ER-модель (database-per-service; PK жирным, FK курсивом-синим)

budget_db budget_plan 1 —< * budget_category

budget_plan	budget_category
id : uuid	id : uuid
owner_id	<i>plan_id → budget_plan</i>
owner_type [USER GROUP]	name
period_start : date	type [MANDATORY DISCRETIONARY]
period_end : date	limit_amount (15,2)
created_at	spent_amount (15,2)

transaction_db

txn
id : uuid
account_id
user_id
amount (15,2)
merchant
mcc
category_name
occurred_at
status
[POSTED DECLINED PENDING_APPROVAL]

goals_db virtual_account 1 —< * goal

virtual_account	goal
id : uuid	id : uuid
owner_id	name
owner_type	target_amount
balance (15,2)	allocated_amount
	monthly_contribution
	deadline : date
	<i>virtual_account_id → va</i>
	bonus_program_ref

family_db family_group 1 —< * membership 1 —o 0..1 child_limit

family_group	membership	child_limit
id : uuid	id : uuid	id : uuid
name	<i>group_id → family_group</i>	<i>membership_id → membership</i>
type = 'FAMILY'	user_id	daily_limit (15,2)
created_by	role [ADULT CHILD]	override_until
created_at	unique(group_id, user_id)	override_amount (15,2)

Между БД (по значению, без FK): owner_id / user_id = Keycloak sub ; owner_type=GROUP ссылается на family_group.id.

9. Categorization Engine (MCC → категория, иначе эвристика по мерчанту, иначе «Прочее»)

5411/5412/5499	5541/5542	4111/4121	5812/5814	4900	7997/7991
Продукты	Транспорт	Такси	Рестораны	Коммуналка	Развлечения

10. Безопасность

- Keycloak realm maqsat : клиент maqsat-app (public, password grant), maqsat-admin (confidential, service-account с realm-management ролями), realm-роль CUSTOMER, юзеры papa/mama/child.
- Каждый сервис — OAuth2 resource server: валидация JWT по issuer-uri, STATELESS, permitAll для actuator/swagger.
- sub = кросс-сервисный userId (через currentUser). KC_HOSTNAME фиксирует iss , чтобы хостовые и in-network токены валидировались одинаково.
- Сервис→сервис: проброс Bearer инициатора.

11. Observability

- **Метрики** — Micrometer → Prometheus (Eureka SD) + кастомные бизнес-счётчики (транзакции по статусу, лимиты, одобрения, трекинг трат).
- **Трейсы** — OTLP → Tempo; один трейс проходит gateway→transaction→Kafka→budget/notification/integration (контекст через Kafka-заголовки). Tempo строит service graph.
- **Логи** — ECS JSON (traceId/spanId), Alloy → Loki. Двусторонняя корреляция trace↔log в Grafana.
- 3 дашборда: Overview, Service Graph & Traces, Business Metrics.

12. Запуск

```
./gradlew build
docker compose up --build                # сборка образов из исходников
docker compose -f docker-compose.yml -f docker-compose.dev.yml up -d            # быстро: готовые jar
docker compose -f ... -f docker-compose.observability.yml up -d                 # + observability
bash demo/run-demo.sh                    # сквозное демо
```

Eureka :8761 · Keycloak :8081 (admin/admin) · Gateway :8080 · Grafana :3000